Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Военный факультет

Кафедра электронных вычислительных машин

ОТЧЕТ
по лабораторной работе № 1
по дисциплине «Жизненный цикл разработки программного обеспечения»
на тему

ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО

Вариант 16

Выполнил студент гр. 434701 Р. А. Пилютик

Проверил О. М. Внук

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Целью лабораторной работы является:

– изучить методы работы с одной из выбранных сред разработки при условии, что используется только объектно-ориентированный язык программирования;

– научиться разрабатывать в выбранной среде приложение с графическим интерфейсом;

– изучить паттерн MVC с активной и пассивной моделью.

Работа выполняется по варианту 16. Реализован основной уровень задания – каркас MVC с активной моделью.

2 ЗАДАНИЕ К ЛАБОРАТОРНОЙ РАБОТЕ

Порядок выполнения работы включает изучение теоретических сведений, ответы на контрольные вопросы, выполнение задания по варианту с демонстрацией всех его элементов преподавателю и оформление отчета. Задание состоит из следующих пунктов:

1 Получить вариант у преподавателя. Выбрать объектно-ориентированный язык программирования и среду разработки, согласовать с преподавателем.

2 Установить и настроить программное обеспечение. Ознакомиться с его функциональными возможностями и средствами отладки. Привести простые примеры работы с функционалом и продемонстрировать работу отладчика в выбранной среде разработки.

3 Создать пустой проект под выданный вариант задания. Установить расширения для работы с SysML v2 и UML; для сред, отличных от MS Visual Studio Code, подобрать аналоги. Результат отобразить в отчете.

4 Разработать и описать в отчете каркас MVC с выданной по варианту моделью.

2.1 Задание по варианту 16

Разработать утилиту по декодированию текста из азбуки Морзе. Пользователь вводит сигналы, закодированные в этой азбуке, программа конвертирует их в текст. На главной форме должна быть кнопка «Ввести данные». По нажатию на нее открывается дополнительное окно для ввода данных. Последние введенные данные нужно запоминать и восстанавливать при повторном вводе. После этого на главной форме отображается результат – упомянутые величины. Если введены некорректные данные – показать сообщение об ошибке. Добавить каркас MVC с активной моделью.

Из формулировки варианта выделены следующие требования к программе:

– ввод сигналов азбуки Морзе;

– преобразование сигналов в текст;

– кнопка «Ввести данные» на главной форме;

– отдельное окно для ввода данных;

– запоминание последних введенных данных и их восстановление при повторном вводе;

– отображение результата на главной форме;

– сообщение об ошибке при некорректных данных;

– каркас MVC с активной моделью.

3 ВЫБОР ЯЗЫКА И СРЕДЫ РАЗРАБОТКИ

В качестве языка программирования выбран объектно-ориентированный язык C# (версия 14), в качестве платформы – .NET 10 с библиотекой графического интерфейса Windows Forms, в качестве среды разработки – Microsoft Visual Studio Community 2026 (версия 18.10). Visual Studio является полноценной интегрированной средой разработки и выступает аналогом сред IntelliJ IDEA, Eclipse и Microsoft Visual Studio Code, перечисленных в задании: в нее входят редактор кода с автодополнением IntelliSense, конструктор форм, обозреватель решений, система сборки MSBuild, отладчик, анализаторы кода и интеграция с системой контроля версий Git [1].

Выбор обоснован тем, что C# – строго типизированный объектно-ориентированный язык с поддержкой интерфейсов и событий, которые напрямую используются при реализации паттерна MVC с активной моделью, а библиотека Windows Forms предоставляет готовые элементы управления: формы, кнопки, поля ввода, переключатели и окна сообщений [2]. Visual Studio изначально ориентирована на разработку приложений .NET и содержит все необходимые инструменты без дополнительной настройки. Выбор языка и среды согласован с преподавателем. Состав инструментального окружения приведен в таблице 1.

Таблица 1 – Состав инструментального окружения

Компонент	**Версия**	**Назначение**
Операционная система	Windows 10 Pro 22H2	Платформа разработки и запуска
Среда разработки Visual Studio Community 2026	18.10.3	Редактирование, сборка, запуск и отладка
Рабочая нагрузка «Разработка классических приложений .NET»	18.10.3	Шаблоны и конструктор Windows Forms, отладчик .NET
Пакет .NET SDK	10.0.401	Компилятор C#, система сборки MSBuild, утилита dotnet
Библиотека Windows Forms	.NET 10	Классы графического интерфейса
Язык программирования	C# 14	Реализация приложения

4 УСТАНОВКА И НАСТРОЙКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1 Установка Visual Studio

Среда Visual Studio Community 2026 установлена с помощью программы Visual Studio Installer. При установке выбрана рабочая нагрузка «Разработка классических приложений .NET», которая включает:

– шаблоны проектов Windows Forms и WPF для .NET и .NET Framework;

– конструктор форм Windows Forms;

– отладчик управляемого кода и средства диагностики;

– пакет .NET SDK и среду выполнения .NET.

Установленные версии пакетов .NET SDK выводятся командой dotnet --list-sdks; для проекта используется версия 10.0.401. Обновления среды устанавливаются через Visual Studio Installer. Главное окно среды с открытым проектом показано на рисунке 1: слева расположен редактор кода, справа – обозреватель решений, внизу – окно «Вывод» с результатом сборки.

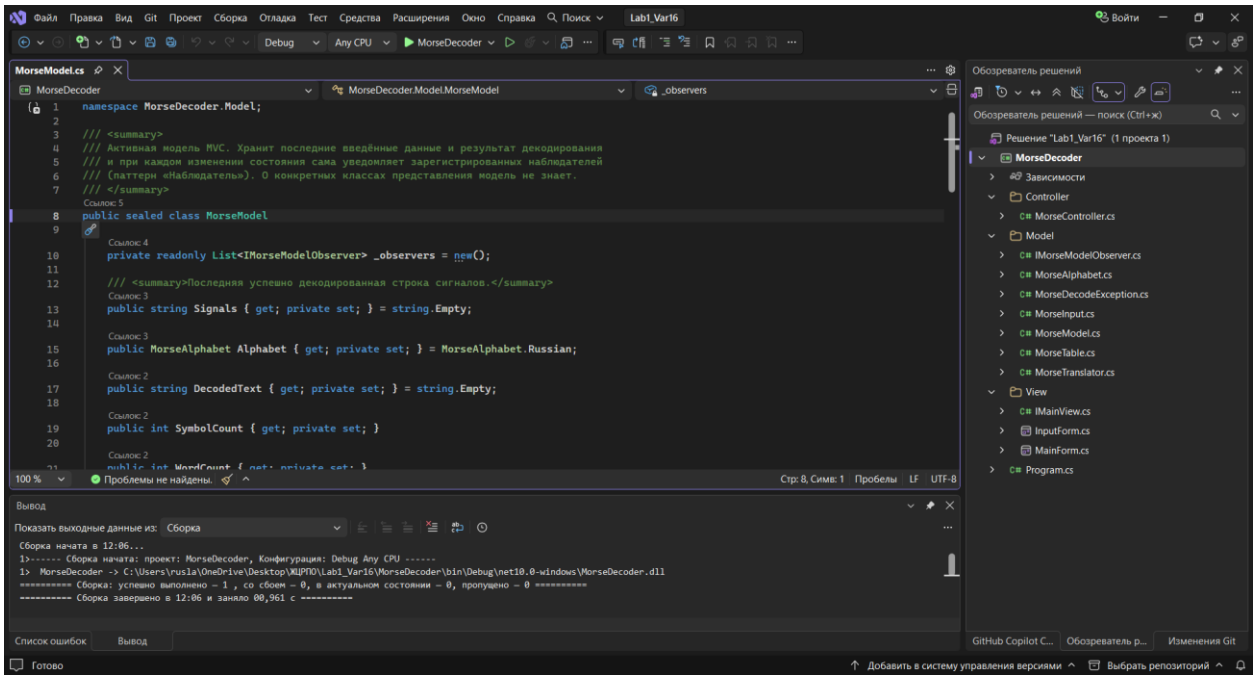


Рисунок 1 – Главное окно Visual Studio с открытым проектом

4.2 Настройка сборки и запуска

Отдельных файлов настройки сборки для проекта не потребовалось: Visual Studio читает описание проекта MorseDecoder.csproj и решения

Lab1_Var16.slnx и сама формирует конфигурации сборки. Используются следующие параметры:

– конфигурация решения Debug и платформа Any CPU, выбираемые на панели инструментов;

– запускаемый проект MorseDecoder (кнопка запуска на панели инструментов);

– каталог вывода bin/Debug/net10.0-windows, в который помещаются сборка MorseDecoder.dll и исполняемый файл MorseDecoder.exe.

Пользовательские настройки среды для решения (открытые файлы, точки останова, состояние окон) Visual Studio хранит в скрытом каталоге .vs рядом с файлом решения. Каталог с лабораторными работами содержит в пути кириллические символы; проверено, что сборка MSBuild и компилятор C# корректно работают с такими путями (окно «Вывод» на рисунке 1 показывает успешную сборку), поэтому дополнительная настройка не потребовалась.

4.3 Функциональные возможности среды

Основные компоненты Visual Studio и их назначение приведены в таблице 2.

Таблица 2 – Основные компоненты среды Visual Studio

Компонент	Назначение и основные возможности
Редактор кода	Подсветка синтаксиса, автодополнение IntelliSense (Ctrl+Пробел), переход к определению (F12), поиск ссылок (Shift+F12), переименование (Ctrl+R, R), быстрые действия (Ctrl+.), форматирование (Ctrl+K, D), CodeLens – число ссылок над членами класса
Обозреватель решений	Структура решения и проектов, файлы, зависимости, добавление элементов из контекстного меню
Конструктор форм Windows Forms	Визуальное проектирование форм, панель элементов, окно свойств (Shift+F7)
Сборка	Сборка решения (Ctrl+Shift+B), конфигурации Debug и Release, окно «Вывод»
Отладчик	Точки останова, пошаговое выполнение, окна «Локальные», «Видимые», «Контрольные значения», «Стек вызовов», «Средства диагностики»
Список ошибок	Ошибки и предупреждения компилятора и анализаторов кода
Анализаторы кода	Правила качества кода (CA) пакета Microsoft.CodeAnalysis.NetAnalyzers
Контроль версий	Окна «Изменения Git» и «Репозиторий Git»
Диспетчер расширений	Поиск и установка расширений из каталога Visual Studio Marketplace
Поиск (Ctrl+Q) и «Перейти ко всему» (Ctrl+T)	Поиск команд среды, файлов, типов и членов проекта

В ходе работы использованы следующие возможности среды:

– формы главного окна и окна ввода созданы программно с помощью контейнеров компоновки TableLayoutPanel и FlowLayoutPanel, что обеспечивает корректное масштабирование окон при разных настройках DPI;

– для навигации по коду использовались переход к определению (F12), поиск ссылок (Shift+F12), подсказки CodeLens и обозреватель решений;

– сборка выполнялась сочетанием Ctrl+Shift+B, запуск – клавишами F5 и Ctrl+F5, сообщения компилятора и анализаторов просматривались в окнах «Вывод» и «Список ошибок».

4.4 Средства отладки

Отладчик Visual Studio запускается клавишей F5 или кнопкой запуска на панели инструментов [3]. Основные приемы работы:

– установка и снятие точки останова – щелчок слева от номера строки или F9;

– шаг с обходом метода – F10, шаг с заходом в метод – F11, выход из метода – Shift+F11, продолжение – F5, остановка отладки – Shift+F5;

– окно «Локальные» показывает локальные переменные текущего метода, окно «Видимые» – переменные, используемые в текущей и предыдущей строках;

– окно «Контрольные значения» вычисляет заданные выражения при каждой остановке;

– окно «Стек вызовов» показывает цепочку вызовов, приведшую к текущей точке;

– при наведении указателя на переменную выводится всплывающая подсказка с ее значением (DataTip);

– в параметрах точки останова задаются условие, число срабатываний и действие (вывод сообщения в окно «Вывод»).

Для демонстрации отладчика точка останова установлена в методе MorseTranslator.Decode() на строке decodedWord.Append(symbol). После запуска под отладчиком в окне ввода введена строка «... --- ...» и выбран латинский алфавит. При первой остановке переменные имели значения code = "..." и symbol = 'S'. После продолжения выполнения (F5) произошла вторая остановка на том же месте (рисунок 2): подсказка рядом с переменной цикла показывает code = "---", переменная symbol равна 'O', а переменная decodedWord, добавленная в окно «Контрольные значения», содержит уже собранную часть слова "S". В окне «Локальные» видны параметры метода signals = "... --- ..." и alphabet = Latin. Окно «Стек вызовов» подтверждает цепочку: обработчик кнопки формы MainForm → MorseController.OnEnterDataRequested() → MorseModel.Decode() → MorseTranslator.Decode().

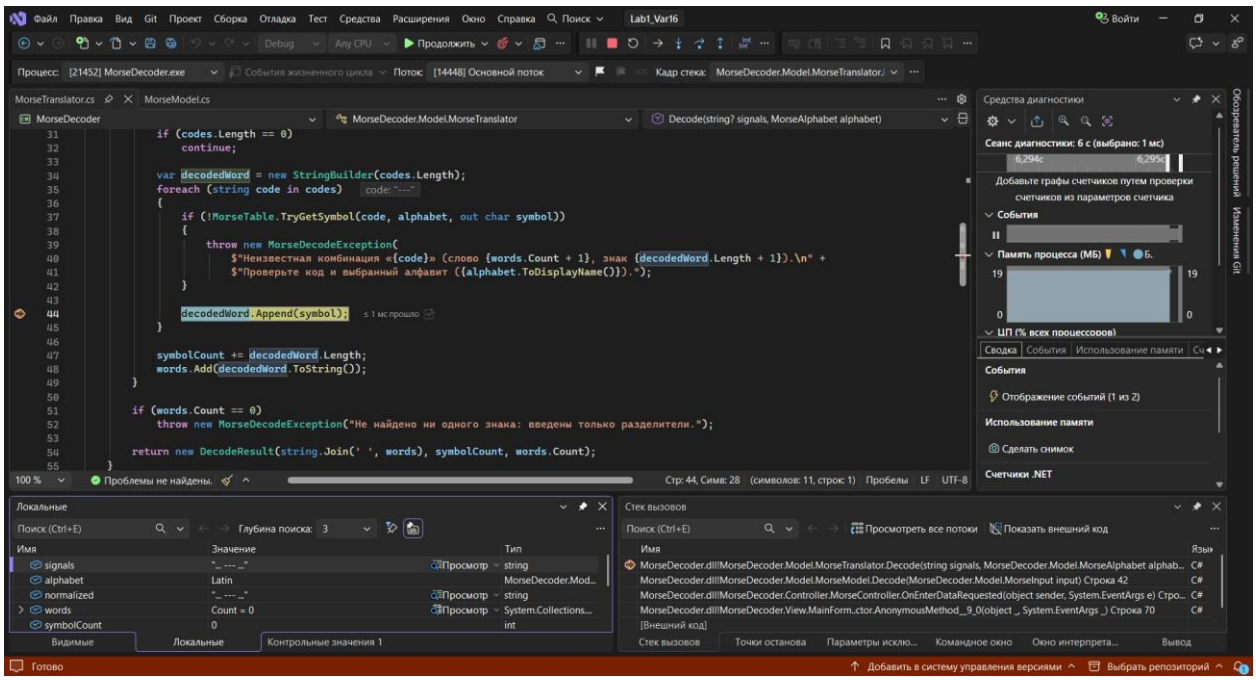


Рисунок 2 – Остановка отладчика в методе MorseTranslator.Decode()

5 СОЗДАНИЕ ПРОЕКТА И РАСШИРЕНИЯ ДЛЯ UML И SYSML V2

5.1 Создание проекта

Проект создан командой меню «Файл → Создать → Проект». В окне «Создание проекта» по запросу «Windows Forms» найден шаблон «Приложение Windows Forms (Майкрософт)» для языка C# (рисунок 3), затем указаны имя проекта MorseDecoder и целевая платформа .NET 10. Решение сохранено в формате Lab1_Var16.slnx. Форма Form1, созданная шаблоном, удалена, а структура проекта приведена в соответствие с паттерном MVC: исходные файлы разнесены по каталогам Model, View и Controller, текстовые модели UML и SysML v2 размещены в каталоге docs (таблица 3).

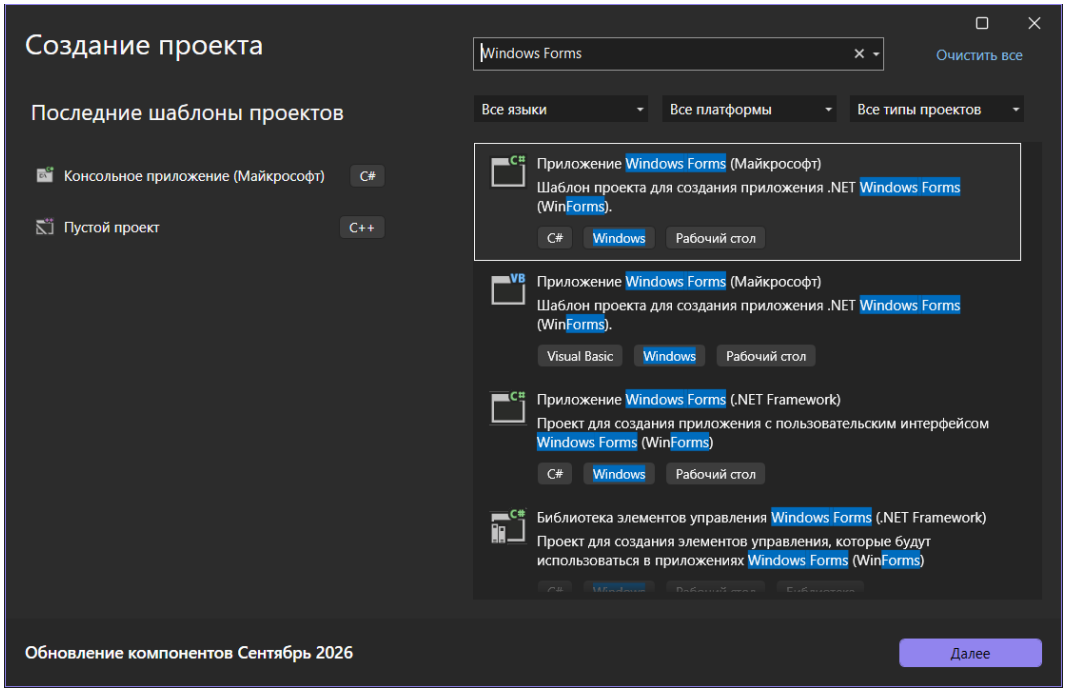


Рисунок 3 – Выбор шаблона в окне «Создание проекта»

Таблица 3 – Структура проекта

Каталог / файл	Содержимое
Lab1_Var16.slnx	Файл решения
MorseDecoder/MorseDecoder.csproj	Описание проекта: тип WinExe, платформа net10.0-windows, Windows Forms
MorseDecoder/Program.cs	Точка входа: создание и связывание слоев MVC
MorseDecoder/Model/	Слой Model: MorseModel, IMorseModelObserver, MorseTranslator, MorseTable, MorseInput, MorseAlphabet, MorseDecodeException
MorseDecoder/View/	Слой View: MainForm, InputForm, IMainView
MorseDecoder/Controller/	Слой Controller: MorseController
docs/*.puml	UML-диаграммы классов и последовательности (PlantUML)
docs/MorseDecoder.sysml	Модель системы в текстовой нотации SysML v2

Полные исходные тексты программы приведены в приложении А.

5.2 Расширения для работы с UML и SysML v2

Примеры расширений в задании приведены для MS Visual Studio Code. Для Visual Studio поиск аналогов выполнен в диспетчере расширений (меню «Расширения → Управление расширениями»), который работает с каталогом Visual Studio Marketplace:

– для UML найдено расширение PlantUML Editor (автор Vincent Dagpin, версия 1.1.133) – редактор диаграмм в текстовой нотации PlantUML [4] с предпросмотром (рисунок 4);

– для SysML v2 расширений под Visual Studio не существует: поиск по запросу «SysML» результатов не дает (рисунок 5); инструменты для SysML v2, например SysIDE компании Sensmetry, выпускаются только для Visual Studio Code.

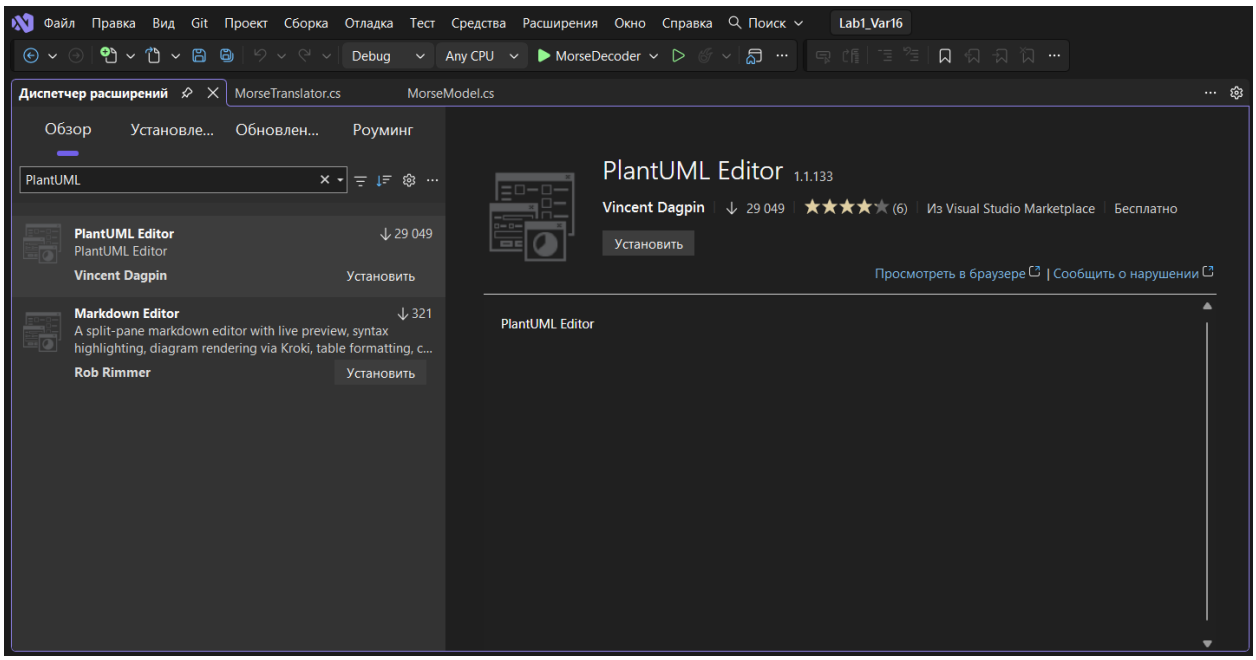


Рисунок 4 – Расширение PlantUML Editor в диспетчере расширений Visual Studio

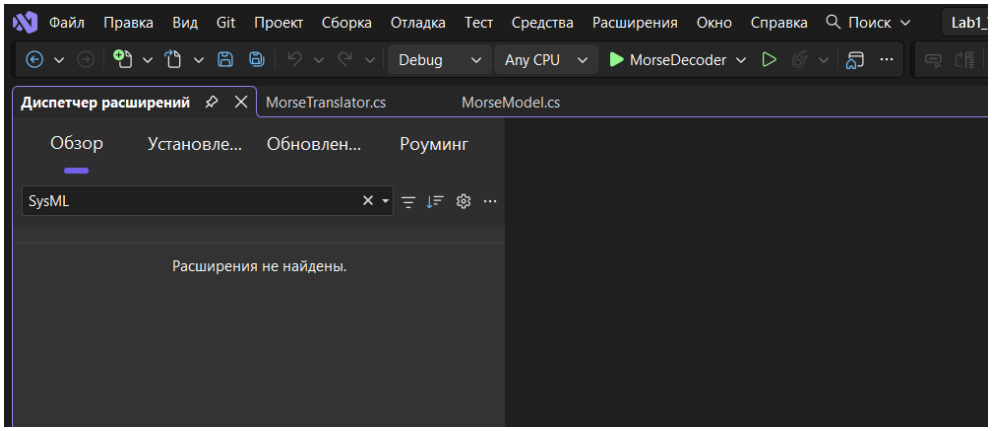


Рисунок 5 – Результат поиска расширений для SysML в диспетчере расширений

Поэтому для SysML v2 в качестве аналога используется текстовая нотация языка [5]: файл модели .sysml хранится в каталоге docs вместе с исходным кодом и открывается в редакторе Visual Studio как текст. Так же хранятся и текстовые описания UML-диаграмм в формате PlantUML.

5.3 UML-модель приложения

Диаграмма классов приложения построена в нотации UML и приведена на рисунке 6. Текстовое описание диаграммы в нотации PlantUML приведено в приложении Б.

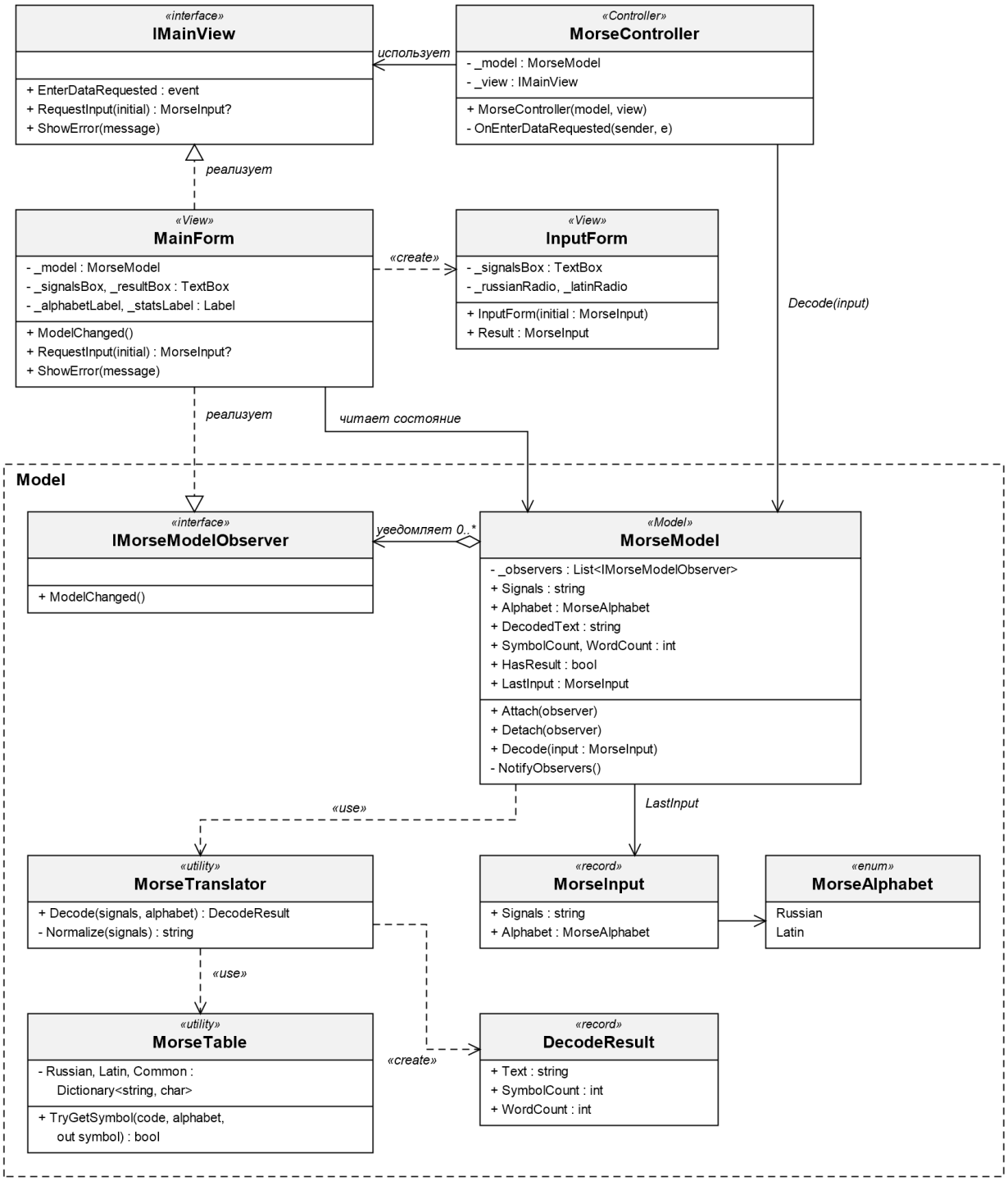


Рисунок 6 – Диаграмма классов приложения

На диаграмме видно разделение классов по слоям: классы с пометкой «View» относятся к представлению, MorseController – к контроллеру, классы в рамке Model – к модели. Класс MainForm реализует интерфейсы IMainView и IMorseModelObserver, а MorseModel хранит список наблюдателей и сам вызывает их метод ModelChanged() – это ключевой признак активной модели. Контроллер зависит не от конкретной формы, а от интерфейса IMainView.

5.4 Модель системы в нотации SysML v2

Модель системы описана в файле docs/MorseDecoder.sysml (приложение Б). В ней определены:

– перечисление MorseAlphabet и типы данных MorseInput и DecodeResult (enum def, attribute def);

– структура системы Application из частей model, mainView, inputView и controller (part def) и их связи через порты ActionPort, DecodePort и NotifyPort;

– поведение DecodeMorse (action def): нормализация ввода, разбиение на слова, поиск кодов в таблице и оповещение наблюдателей;

– требования варианта (requirement def) и их удовлетворение частью variant16 (satisfy).

Вид модели в редакторе Visual Studio показан на рисунке 7.

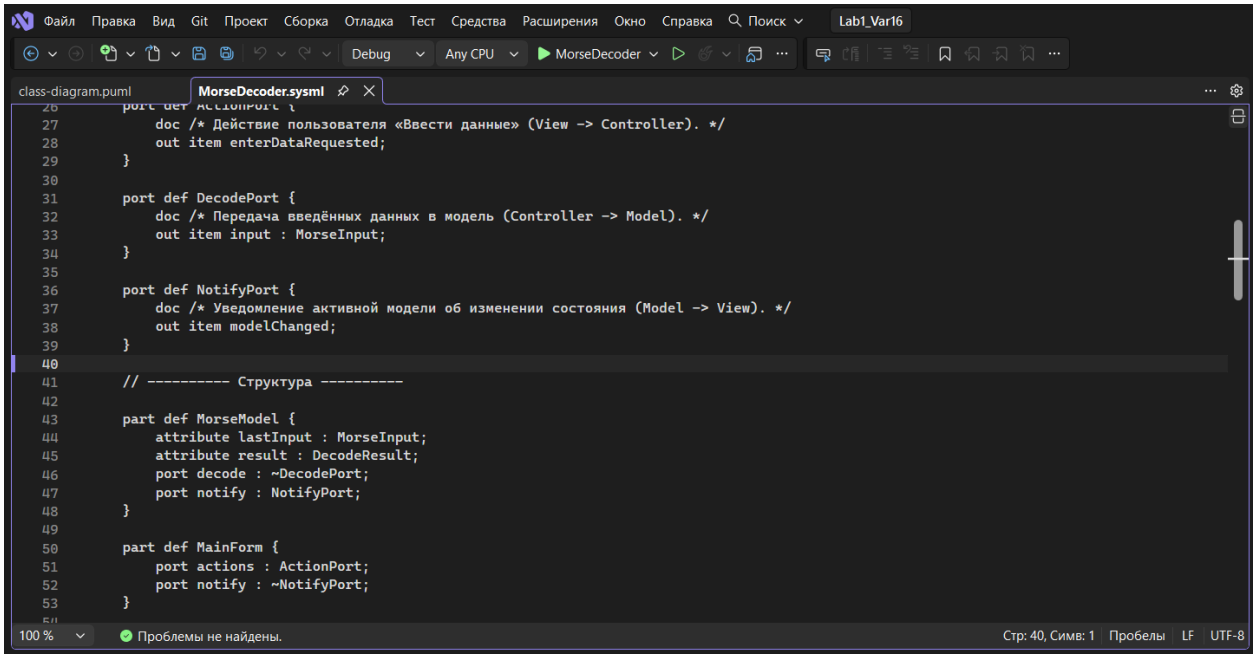


Рисунок 7 – Модель системы в нотации SysML v2 в редакторе Visual Studio

6 РАЗРАБОТКА КАРКАСА MVC С АКТИВНОЙ МОДЕЛЬЮ

6.1 Общая структура

Приложение построено по паттерну MVC (Model – View – Controller), который разделяет программу на три слоя: модель хранит данные и бизнес-логику, представление отображает данные, контроллер обрабатывает действия пользователя [6]. По варианту требуется активная модель: модель сама оповещает представления об изменении своего состояния с помощью паттерна «Наблюдатель» (Observer), ничего не зная о конкретных классах представлений. Схема взаимодействия слоев приведена на рисунке 8.

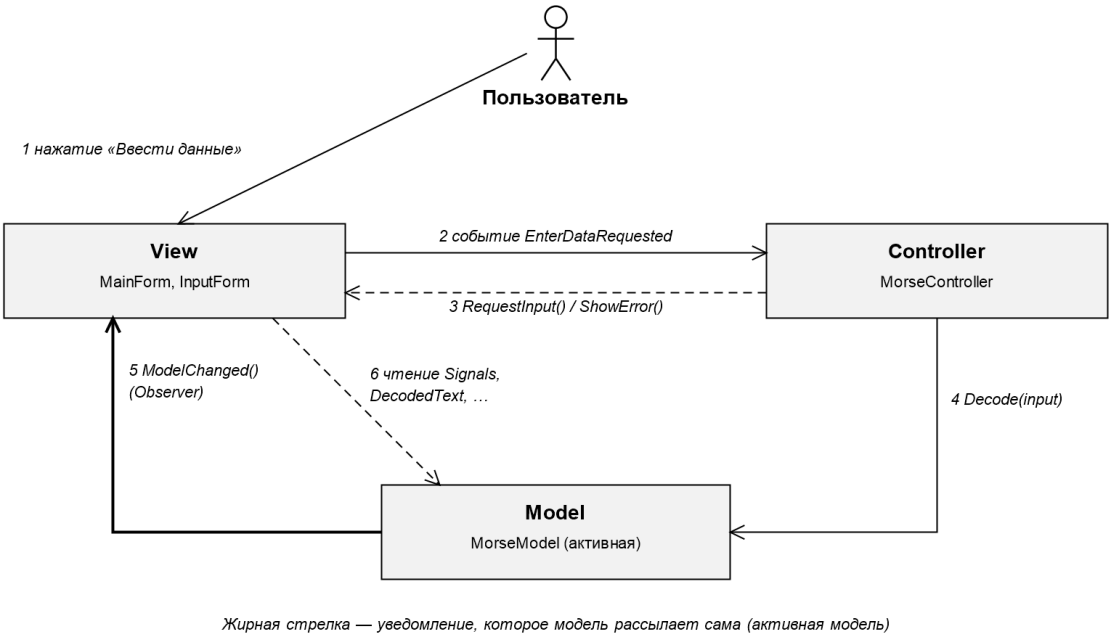


Рисунок 8 – Схема взаимодействия слоев MVC с активной моделью

Все три слоя создаются и связываются в одном месте – методе Main() класса Program (файл Program.cs): сначала создается модель, затем главное окно, получающее ссылку на модель, и контроллер, получающий ссылки на модель и на главное окно через интерфейс IMainView.

6.2 Слой Model

Слой модели не зависит от классов Windows Forms и включает следующие элементы:

– IMorseModelObserver – интерфейс наблюдателя с методом ModelChanged();

– MorseModel – активная модель: хранит последние введенные данные (Signals, Alphabet) и результат декодирования (DecodedText, SymbolCount, WordCount), регистрирует наблюдателей методами Attach() и Detach() и рассылает им уведомления;

– MorseTranslator – алгоритм декодирования и проверки ввода, вынесенный из модели в отдельный класс;

– MorseTable – таблицы кодов азбуки Морзе [7]: русский алфавит (32 кода, буква Ё передается кодом буквы Е), латинский алфавит (26 кодов), а также общие для обоих алфавитов цифры (10 кодов) и знаки препинания (18 кодов);

– MorseInput – данные, введенные пользователем: строка сигналов и выбранный алфавит;

– DecodeResult – результат декодирования: текст, количество знаков и слов;

– MorseDecodeException – исключение, сообщающее о некорректных данных.

Единственная точка входа в модель для контроллера – метод Decode(). Он передает данные декодеру MorseTranslator и только при успешном декодировании сохраняет ввод и результат, после чего сам вызывает метод NotifyObservers(), рассылающий уведомление ModelChanged() всем подписанным наблюдателям. При некорректных данных декодер бросает исключение MorseDecodeException, состояние модели не меняется, и уведомление не рассылается.

Декодирование выполняется в три этапа. Сначала метод Normalize() приводит строку к единому виду: точками считаются символы «.», «·» и «•», тире – «-», «−», «–», «—» и «_», разделителями слов – «/», «|» и перевод строки; любой другой символ считается ошибкой. Затем строка разбивается на слова по косой черте или по двум и более пробелам подряд, а каждое слово – на коды знаков по одиночным пробелам. Наконец, каждый код ищется сначала в таблице выбранного алфавита, затем в таблице цифр и знаков препинания. Проверки корректности данных приведены в таблице 4.

Таблица 4 – Проверки корректности входных данных

Ситуация	Сообщение об ошибке
Пустая строка или строка из одних пробелов	Строка сигналов пуста. Введите код азбуки Морзе, например: ... --- ...
Символ, отличный от точки, тире, пробела и разделителей слов	Недопустимый символ «S» в позиции 1. Разрешены только точка (.), тире (-), пробел и косая черта (/).
Комбинация, отсутствующая в таблице выбранного алфавита	Неизвестная комбинация «.....» (слово 1, знак 2). Проверьте код и выбранный алфавит (латинский).
Введены только разделители слов	Не найдено ни одного знака: введены только разделители.

6.3 Слой View

Слой представления состоит из следующих элементов:

– IMainView – интерфейс главного представления, через который с ним работает контроллер: событие EnterDataRequested и методы RequestInput() и ShowError();

– MainForm – главная форма с кнопкой «Ввести данные», полями «Введенные сигналы» и «Расшифрованный текст», меткой выбранного алфавита и меткой с количеством знаков и слов; класс реализует интерфейсы IMainView и IMorseModelObserver, в конструкторе подписывается на модель вызовом Attach(this), при закрытии формы отписывается вызовом Detach(this);

– InputForm – модальное окно ввода с многострочным полем для сигналов, переключателем алфавита (русский или латинский) и кнопками «Пример», «Декодировать» и «Отмена»; поля заполняются последними введенными данными, полученными из модели.

Получив уведомление ModelChanged(), главная форма сама читает свойства модели и обновляет поля. Вычислений и проверок в представлении нет – оно только отображает состояние модели и сообщает контроллеру о нажатии кнопки. Поля с сигналами и результатом сделаны многострочными полями только для чтения, а не метками, чтобы длинный текст переносился по строкам и его можно было скопировать.

6.4 Слой Controller

Контроллер MorseController в конструкторе подписывается на событие EnterDataRequested главной формы. Обработчик события OnEnterDataRequested() получает у модели последние введенные данные

(свойство LastInput), вызывает метод представления RequestInput(), который открывает окно InputForm, заполненное этими данными, и передает введенное в модель методом Decode(). Если модель отклоняет данные исключением MorseDecodeException, контроллер вызывает метод ShowError() и снова открывает окно ввода с ошибочной строкой, чтобы пользователь мог ее исправить. Отображение результата контроллер не выполняет – главная форма обновляется по уведомлению модели, что и отличает схему с активной моделью.

6.5 Взаимодействие компонентов

Последовательность обработки нажатия кнопки «Ввести данные» приведена на диаграмме последовательности (рисунок 9).

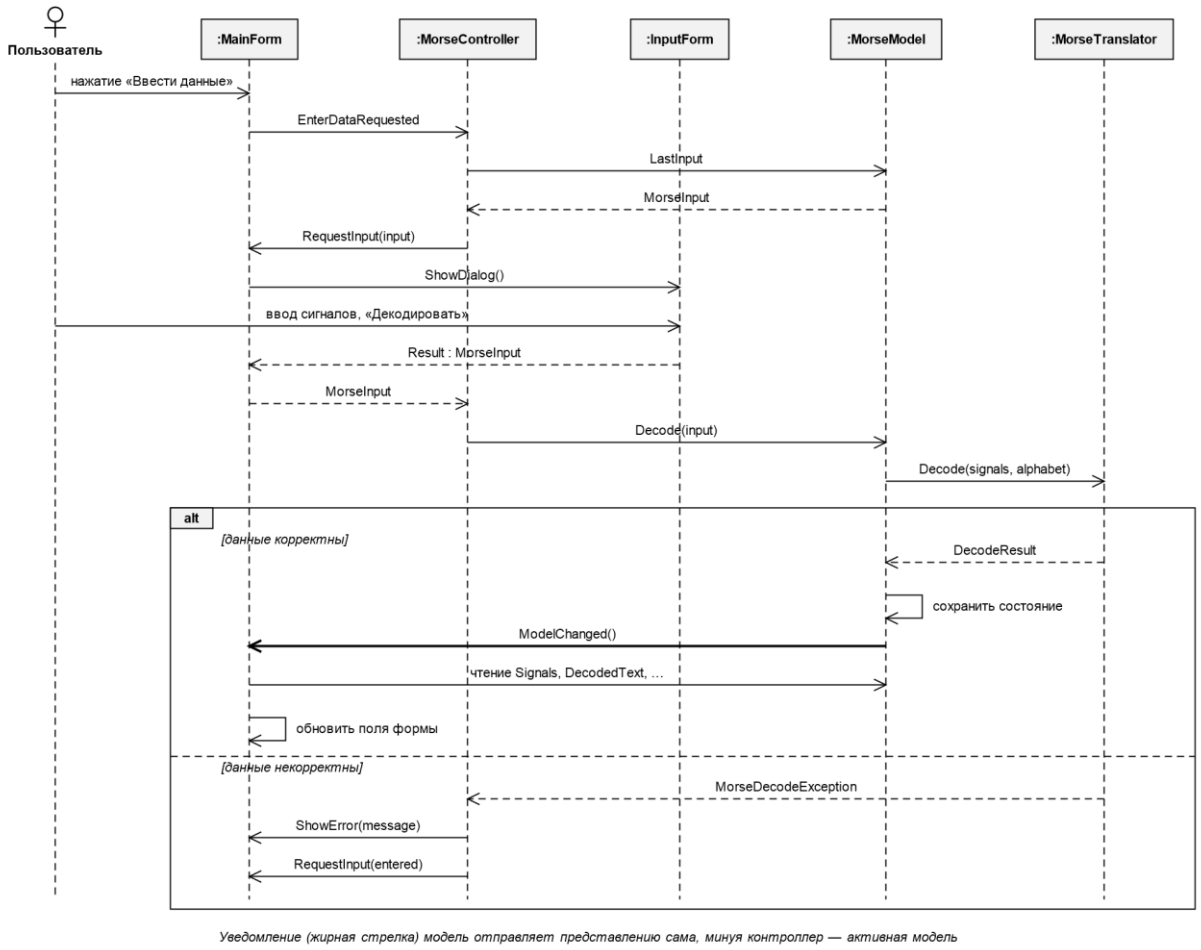


Рисунок 9 – Диаграмма последовательности обработки ввода данных

Пользователь нажимает кнопку, главная форма генерирует событие EnterDataRequested, контроллер получает у модели последний ввод и через представление открывает окно ввода. После нажатия кнопки «Декодировать» данные передаются в модель, которая вызывает декодер. При корректных данных модель сохраняет состояние и сама вызывает метод ModelChanged() главной формы, после чего форма читает из модели новые значения. При некорректных данных декодер бросает исключение, контроллер показывает сообщение об ошибке и повторно открывает окно ввода.

6.6 Отличие активной модели от пассивной

В схеме с пассивной моделью модель не знает о представлениях и не рассылает уведомлений: после изменения состояния контроллер сам вызывает методы представления для его обновления. Чтобы превратить разработанный каркас в схему с пассивной моделью, потребовалось бы удалить из MorseModel список наблюдателей, методы Attach(), Detach(), NotifyObservers() и интерфейс IMorseModelObserver, а в интерфейс IMainView добавить метод, например ShowResult(), который контроллер вызывал бы после успешного вызова Decode().

Активная модель удобнее, когда одно состояние отображается несколькими представлениями: новое представление достаточно подписать на модель, не изменяя ни модель, ни контроллер.

6.7 Результаты работы программы

При запуске программы отображается главная форма с пустыми полями результата и подсказкой (рисунок 10).

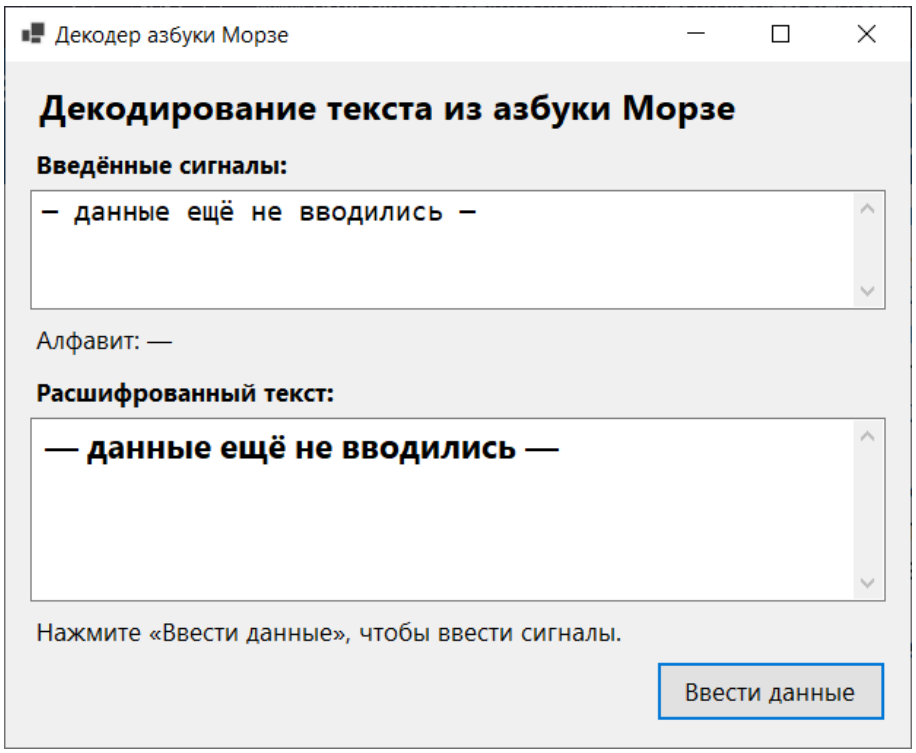


Рисунок 10 – Главная форма при запуске программы

По нажатию кнопки «Ввести данные» открывается окно ввода (рисунок 11). При повторном открытии поле сигналов и переключатель алфавита заполнены последними введенными данными. Кнопка «Пример» вставляет пример строки для выбранного алфавита.

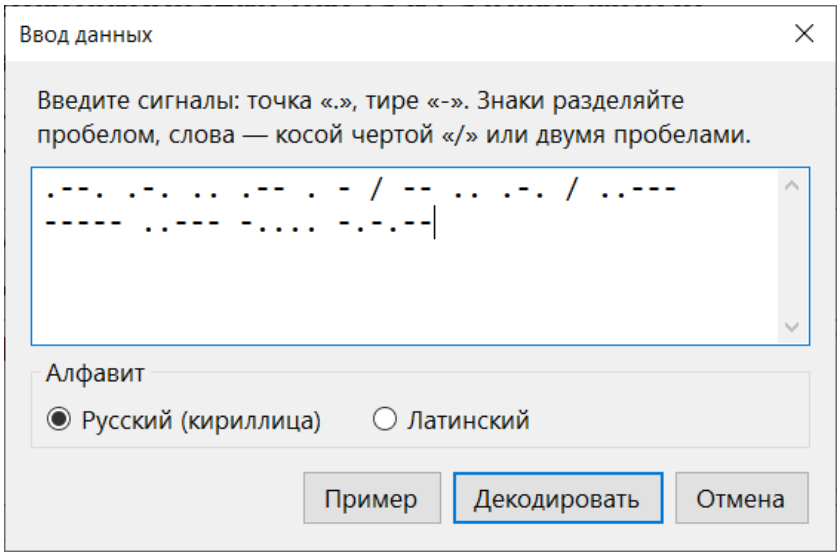


Рисунок 11 – Окно ввода данных с восстановленными значениями

После нажатия кнопки «Декодировать» на главной форме отображаются введенные сигналы, выбранный алфавит, расшифрованный текст и количество знаков и слов (рисунки 12 и 13).

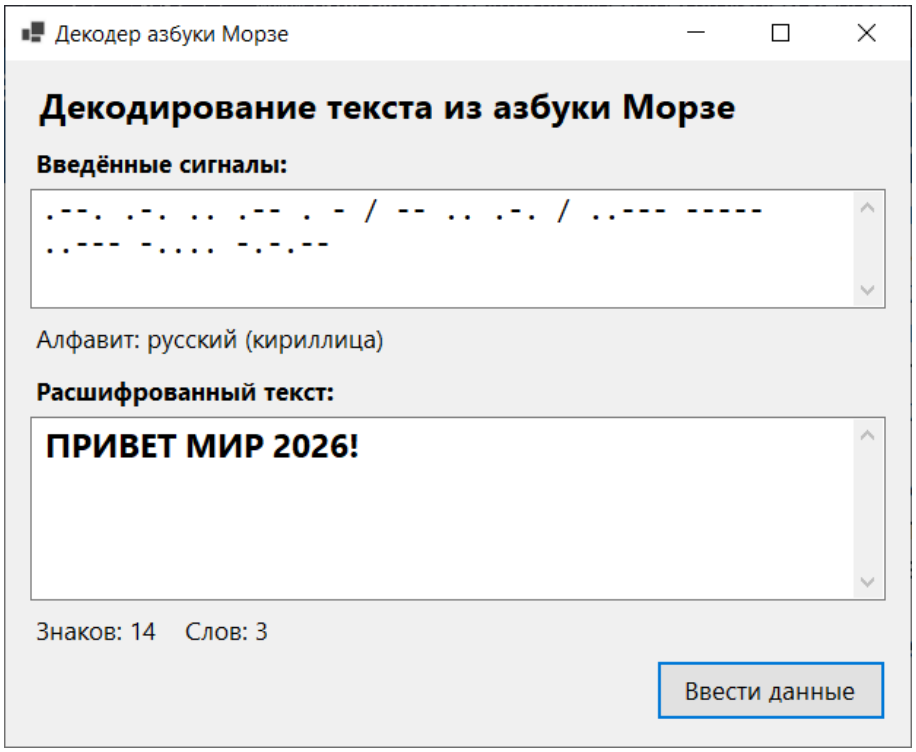


Рисунок 12 – Результат декодирования текста на русском языке

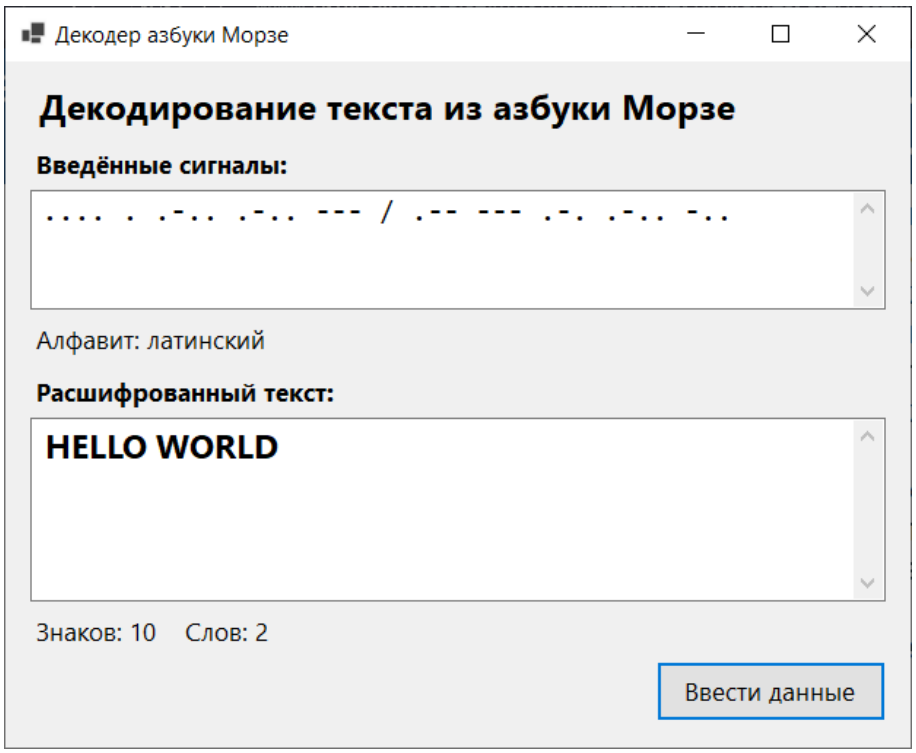


Рисунок 13 – Результат декодирования текста на латинице

При вводе некорректных данных выводится сообщение об ошибке с пояснением причины (рисунок 14).

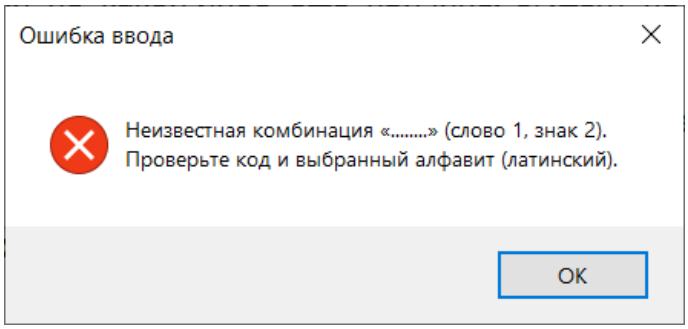


Рисунок 14 – Сообщение об ошибке при неизвестной комбинации сигналов

Результаты тестирования программы на корректных и некорректных данных приведены в таблице 5.

Таблица 5 – Результаты тестирования программы

Входная строка	Алфавит	Результат
.--. .-. .. .-- . - / -- .. .-.	русский	ПРИВЕТ МИР (9 знаков, 2 слова)
.... . .-.. .-.. --- / .-- --- .-. .-.. -..	латинский	HELLO WORLD (10 знаков, 2 слова)
... --- ...	латинский	SOS
..--- ----- ..--- -.... / -.-.--	русский	2026 !
-- --- .-. --.. . ↵ .- --.. -... ..- -.- .-	русский	МОРЗЕ АЗБУКА (слова разделены переводом строки)
·-- ·-· --- --	русский	ВР ОМ (другие символы точки и тире, слова разделены пробелами)
(пустая строка)	любой	Сообщение «Строка сигналов пуста»
SOS	латинский	Сообщение «Недопустимый символ «S» в позиции 1»
... .	латинский	Сообщение «Неизвестная комбинация «.....»»
---- / ..-..	латинский	Сообщение «Неизвестная комбинация «----»»: в латинице такого кода нет
/ /	русский	Сообщение «Не найдено ни одного знака»

6.8 Соответствие программы требованиям варианта

Соответствие разработанной программы требованиям варианта 16 приведено в таблице 6.

Таблица 6 – Выполнение требований варианта

Требование	**Реализация**
Ввод сигналов азбуки Морзе	Многострочное поле _signalsBox окна InputForm
Преобразование сигналов в текст	Метод MorseTranslator.Decode(), таблицы MorseTable
Кнопка «Ввести данные» на главной форме	Кнопка формы MainForm, событие EnterDataRequested
Отдельное окно для ввода	Модальное окно InputForm
Запоминание последних данных	Свойства Signals и Alphabet модели, заполняются в Decode()
Восстановление при повторном вводе	Окно InputForm создается с данными model.LastInput
Отображение результата на главной форме	Метод ModelChanged() формы MainForm
Сообщение об ошибке	MorseDecodeException → MorseController → ShowError() → MessageBox
Каркас MVC с активной моделью	Интерфейс IMorseModelObserver, методы Attach(), Detach(), NotifyObservers() модели

7 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1 Какие компоненты входят в состав IDE?

В состав IDE входят редактор кода с подсветкой синтаксиса и автодополнением, компилятор или интерпретатор и система сборки, отладчик, средства навигации и рефакторинга, менеджер проектов (в Visual Studio – обозреватель решений), средства визуального проектирования интерфейса (в Visual Studio – конструктор форм Windows Forms), менеджер пакетов (для .NET – NuGet), интеграция с системой контроля версий, средства тестирования, профилирования и анализа кода, система расширений.

2 Каковы основные приемы работы с редактором кода?

Основные приемы: автодополнение IntelliSense (Ctrl+Пробел), переход к определению (F12) и поиск ссылок (Shift+F12), переименование символа во всем решении (Ctrl+R, R), быстрые действия и рефакторинг (Ctrl+.), поиск и замена в файле (Ctrl+F, Ctrl+H) и по решению (Ctrl+Shift+F), «Перейти ко всему» (Ctrl+T), закомментирование и раскомментирование строк (Ctrl+K, C и Ctrl+K, U), форматирование документа (Ctrl+K, D), сворачивание блоков (Ctrl+M, M), фрагменты кода (snippets) и подсказки CodeLens.

3 Каким образом выполняется сборка и запуск проекта?

Проект описан в файле MorseDecoder.csproj, где указаны тип приложения (WinExe), целевая платформа net10.0-windows и подключение Windows Forms. Команда «Сборка → Собрать решение» (Ctrl+Shift+B) восстанавливает пакеты NuGet, вызывает MSBuild и компилятор C# и помещает сборку MorseDecoder.dll и исполняемый файл MorseDecoder.exe в каталог bin/Debug/net10.0-windows; ход сборки выводится в окно «Вывод». Запуск под отладчиком выполняется клавишей F5, без отладки – сочетанием Ctrl+F5. Из командной строки те же действия выполняют команды dotnet build и dotnet run.

4 Что такое MVC?

MVC (Model – View – Controller) – архитектурный паттерн, разделяющий приложение на три слоя: модель (данные и бизнес-логика), представление (отображение данных) и контроллер (обработка действий пользователя). Разделение снижает связанность: логику можно изменять и тестировать независимо от интерфейса, а интерфейс – заменять без изменения логики.

5 Чем отличается активная модель от пассивной?

В пассивной модели модель не знает о представлениях; после изменения ее состояния представление обновляет контроллер. В активной модели модель хранит список подписчиков и сама рассылает им уведомления, не зная конкретных классов представлений; контроллер в обновлении экрана не участвует.

ВЫВОД

В ходе лабораторной работы выбрана система программирования: язык C#, платформа .NET 10 с библиотекой Windows Forms и среда Visual Studio Community 2026. Изучены функциональные возможности среды и средства отладки; работа отладчика продемонстрирована на примере остановок в методе декодирования с просмотром локальных переменных, контрольных значений и стека вызовов.

Создан проект под вариант 16. Для работы с UML в каталоге Visual Studio Marketplace подобрано расширение PlantUML Editor; расширений для SysML v2 под Visual Studio не существует, поэтому в качестве аналога используется текстовая нотация SysML v2, хранимая в проекте. Построены диаграмма классов, диаграмма последовательности и модель системы в нотации SysML v2.

Разработана утилита декодирования текста из азбуки Морзе по схеме MVC с активной моделью: модель самостоятельно оповещает главную форму об изменении состояния через интерфейс наблюдателя, контроллер только передает данные модели. Программа поддерживает русский и латинский алфавиты, цифры и знаки препинания и выполняет все требования варианта: ввод данных в отдельном окне, запоминание и восстановление последних данных, отображение результата на главной форме и вывод сообщений об ошибках.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Документация по Visual Studio [Электронный ресурс]. – Режим доступа : https://learn.microsoft.com/ru-ru/visualstudio/.

[2] Документация по Windows Forms для .NET [Электронный ресурс]. – Режим доступа : https://learn.microsoft.com/ru-ru/dotnet/desktop/winforms/.

[3] Первое знакомство с отладчиком Visual Studio [Электронный ресурс]. – Режим доступа : https://learn.microsoft.com/ru-ru/visualstudio/debugger/debugger-feature-tour.

[4] PlantUML [Электронный ресурс]. – Режим доступа : https://plantuml.com/.

[5] OMG Systems Modeling Language (SysML) Version 2 [Электронный ресурс]. – Режим доступа : https://www.omg.org/sysml/sysmlv2/.

[6] Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма [и др.]. – СПб. : Питер, 2001. – 368 с.

[7] Recommendation ITU-R M.1677-1. International Morse code [Электронный ресурс]. – Geneva : ITU, 2009. – Режим доступа : https://www.itu.int/rec/R-REC-M.1677-1-200910-I/.

[8] Жизненный цикл разработки программного обеспечения : лабораторная работа № 1 «Интегрированная среда разработки ПО» [Электронный ресурс]. – Минск : БГУИР, 2026.

ПРИЛОЖЕНИЕ А
(обязательное)
Исходные тексты программы

Файл MorseDecoder/MorseDecoder.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net10.0-windows</TargetFramework>
    <UseWindowsForms>true</UseWindowsForms>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <RootNamespace>MorseDecoder</RootNamespace>
    <AssemblyName>MorseDecoder</AssemblyName>
  </PropertyGroup>

</Project>
```

Файл MorseDecoder/Program.cs

```
using MorseDecoder.Controller;
using MorseDecoder.Model;
using MorseDecoder.View;

namespace MorseDecoder;

internal static class Program
{
    [STAThread]
    private static void Main()
    {
        ApplicationConfiguration.Initialize();

        // Сборка триады MVC:
        // модель ничего не знает о представлении и контроллере (только об
интерфейсе наблюдателя),
        // представление подписывается на модель, контроллер связывает
действия пользователя с моделью.
        var model = new MorseModel();
        var view = new MainForm(model);
        _ = new MorseController(model, view);

        Application.Run(view);
    }
}
```

Файл MorseDecoder/Model/IMorseModelObserver.cs

```
namespace MorseDecoder.Model;

/// <summary>
/// Наблюдатель активной модели. Модель сама оповещает подписчиков об
изменении
```

```
/// своего состояния, после чего они читают из неё актуальные данные.
/// </summary>
public interface IMorseModelObserver
{
    void ModelChanged();
}
```

Файл MorseDecoder/Model/MorseAlphabet.cs

```
namespace MorseDecoder.Model;

/// <summary>Алфавит, по которому расшифровываются буквенные коды.</summary>
public enum MorseAlphabet
{
    Russian,
    Latin
}

public static class MorseAlphabetExtensions
{
    public static string ToDisplayName(this MorseAlphabet alphabet) =>
alphabet switch
    {
        MorseAlphabet.Russian => "русский (кириллица)",
        MorseAlphabet.Latin => "латинский",
        _ => alphabet.ToString()
    };
}
```

Файл MorseDecoder/Model/MorseInput.cs

```
namespace MorseDecoder.Model;

/// <summary>Данные, вводимые пользователем: строка сигналов и выбранный
алфавит.</summary>
public sealed record MorseInput(string Signals, MorseAlphabet Alphabet);
```

Файл MorseDecoder/Model/MorseDecodeException.cs

```
namespace MorseDecoder.Model;

/// <summary>Ошибка разбора введённых сигналов (некорректные
данные).</summary>
public sealed class MorseDecodeException : Exception
{
    public MorseDecodeException(string message) : base(message)
    {
    }
}
```

Файл MorseDecoder/Model/MorseTable.cs

```
namespace MorseDecoder.Model;

/// <summary>Таблицы кодов азбуки Морзе: код → символ.</summary>
```

```
public static class MorseTable
{
    private static readonly Dictionary<string, char> Russian = new()
    {
        [".-"] = 'А', ["-..."] = 'Б', [".--"] = 'В', ["--."] = 'Г', ["-.."] =
'Д',
        ["."] = 'Е', ["...-"] = 'Ж', ["--.."] = 'З', [".."] = 'И', [".---"] =
'Й',
        ["-.-"] = 'К', [".-.."] = 'Л', ["--"] = 'М', ["-."] = 'Н', ["---"] =
'О',
        [".--."] = 'П', [".-."] = 'Р', ["..."] = 'С', ["-"] = 'Т', ["..-"] =
'У',
        ["..-."] = 'Ф', ["...."] = 'Х', ["-.-."] = 'Ц', ["---."] = 'Ч', ["---
-"] = 'Ш',
        ["--.-"] = 'Щ', ["--.--"] = 'Ъ', ["-.--"] = 'Ы', ["-..-"] = 'Ь',
["..-.."] = 'Э',
        ["..--"] = 'Ю', [".-.-"] = 'Я'
    };

    private static readonly Dictionary<string, char> Latin = new()
    {
        [".-"] = 'A', ["-..."] = 'B', ["-.-."] = 'C', ["-.."] = 'D', ["."] =
'E',
        ["..-."] = 'F', ["--."] = 'G', ["...."] = 'H', [".."] = 'I', [".---"]
= 'J',
        ["-.-"] = 'K', [".-.."] = 'L', ["--"] = 'M', ["-."] = 'N', ["---"] =
'O',
        [".--."] = 'P', ["--.-"] = 'Q', [".-."] = 'R', ["..."] = 'S', ["-"] =
'T',
        ["..-"] = 'U', ["...-"] = 'V', [".--"] = 'W', ["-..-"] = 'X', ["-.--
"] = 'Y',
        ["--.."] = 'Z'
    };

    /// <summary>Цифры и знаки препинания — общие для обоих
алфавитов.</summary>
    private static readonly Dictionary<string, char> Common = new()
    {
        ["-----"] = '0', [".----"] = '1', ["..---"] = '2', ["...--"] = '3',
["....-"] = '4',
        ["....."] = '5', ["-...."] = '6', ["--..."] = '7', ["---.."] = '8',
["----."] = '9',
        [".-.-.-"] = '.', ["--..--"] = ',', ["..--.."] = '?', [".----."] =
'\'', ["-.-.--"] = '!',
        ["-..-."] = '/', ["-.--."] = '(', ["-.--.-"] = ')', [".-..."] = '&',
["---..."] = ':',
        ["-.-.-."] = ';', ["-...-"] = '=', [".-.-."] = '+', ["-....-"] = '-',
["..--.-"] = '_',
        [".-..-."] = '"', ["...-..-"] = '$', [".--.-."] = '@'
    };

    public static bool TryGetSymbol(string code, MorseAlphabet alphabet, out
char symbol)
    {
        var letters = alphabet == MorseAlphabet.Russian ? Russian : Latin;
        return letters.TryGetValue(code, out symbol) ||
Common.TryGetValue(code, out symbol);
    }
}
```

Файл MorseDecoder/Model/MorseTranslator.cs

```
using System.Text;
using System.Text.RegularExpressions;

namespace MorseDecoder.Model;

/// <summary>Результат декодирования.</summary>
public sealed record DecodeResult(string Text, int SymbolCount, int
WordCount);

/// <summary>
/// Декодер азбуки Морзе.
/// Формат ввода: точка «.» и тире «-», знаки разделяются пробелом,
/// слова — косой чертой «/» (или «|», переводом строки, двумя и более
пробелами).
/// </summary>
public static class MorseTranslator
{
    private static readonly Regex WordSeparator = new(@"\s*/\s*| {2,}",
RegexOptions.Compiled);

    public static DecodeResult Decode(string? signals, MorseAlphabet
alphabet)
    {
        if (string.IsNullOrWhiteSpace(signals))
            throw new MorseDecodeException("Строка сигналов пуста.\nВведите
код азбуки Морзе, например: ... --- ...");

        string normalized = Normalize(signals).Trim();

        var words = new List<string>();
        int symbolCount = 0;

        foreach (string word in WordSeparator.Split(normalized))
        {
            string[] codes = word.Split(' ',
StringSplitOptions.RemoveEmptyEntries);
            if (codes.Length == 0)
                continue;

            var decodedWord = new StringBuilder(codes.Length);
            foreach (string code in codes)
            {
                if (!MorseTable.TryGetSymbol(code, alphabet, out char
symbol))
                {
                    throw new MorseDecodeException(
                        $"Неизвестная комбинация «{code}» (слово {words.Count
+ 1}, знак {decodedWord.Length + 1}).\n" +
                        $"Проверьте код и выбранный алфавит
({alphabet.ToDisplayName()}).");
                }

                decodedWord.Append(symbol);
            }

            symbolCount += decodedWord.Length;
            words.Add(decodedWord.ToString());
        }
```

```
        if (words.Count == 0)
            throw new MorseDecodeException("Не найдено ни одного знака:
введены только разделители.");

        return new DecodeResult(string.Join(' ', words), symbolCount,
words.Count);
    }

    /// <summary>
    /// Приводит альтернативные обозначения точки/тире к «.» и «-», а
разделители слов — к «/».
    /// Любой другой символ считается ошибкой ввода.
    /// </summary>
    private static string Normalize(string signals)
    {
        var result = new StringBuilder(signals.Length);

        for (int i = 0; i < signals.Length; i++)
        {
            char c = signals[i];
            switch (c)
            {
                case '.' or '·' or '•':
                    result.Append('.');
                    break;
                case '-' or '‐' or '−' or '—' or '_':
                    result.Append('-');
                    break;
                case ' ' or '\t' or ' ':
                    result.Append(' ');
                    break;
                case '/' or '|' or '\n':
                    result.Append('/');
                    break;
                case '\r':
                    break;
                default:
                    throw new MorseDecodeException(
                        $"Недопустимый символ «{c}» в позиции {i + 1}.\n" +
                        "Разрешены только точка (.), тире (-), пробел и косая
черта (/).");
            }
        }

        return result.ToString();
    }
}
```

Файл MorseDecoder/Model/MorseModel.cs

```
namespace MorseDecoder.Model;

/// <summary>
/// Активная модель MVC. Хранит последние введённые данные и результат
декодирования
/// и при каждом изменении состояния сама уведомляет зарегистрированных
наблюдателей
/// (паттерн «Наблюдатель»). О конкретных классах представления модель не
знает.
/// </summary>
```

```
public sealed class MorseModel
{
    private readonly List<IMorseModelObserver> _observers = new();

    /// <summary>Последняя успешно декодированная строка сигналов.</summary>
    public string Signals { get; private set; } = string.Empty;

    public MorseAlphabet Alphabet { get; private set; } =
MorseAlphabet.Russian;

    public string DecodedText { get; private set; } = string.Empty;

    public int SymbolCount { get; private set; }

    public int WordCount { get; private set; }

    public bool HasResult { get; private set; }

    /// <summary>Последние введённые данные — для восстановления в окне
ввода.</summary>
    public MorseInput LastInput => new(Signals, Alphabet);

    public void Attach(IMorseModelObserver observer)
    {
        if (!_observers.Contains(observer))
            _observers.Add(observer);
    }

    public void Detach(IMorseModelObserver observer) =>
_observers.Remove(observer);

    /// <summary>
    /// Декодирует сигналы и обновляет состояние модели.
    /// При некорректных данных бросает <see cref="MorseDecodeException"/>,
состояние не меняется.
    /// </summary>
    public void Decode(MorseInput input)
    {
        DecodeResult result = MorseTranslator.Decode(input.Signals,
input.Alphabet);

        Signals = input.Signals.Trim();
        Alphabet = input.Alphabet;
        DecodedText = result.Text;
        SymbolCount = result.SymbolCount;
        WordCount = result.WordCount;
        HasResult = true;

        NotifyObservers();
    }

    private void NotifyObservers()
    {
        foreach (IMorseModelObserver observer in _observers.ToArray())
            observer.ModelChanged();
    }
}
```

Файл MorseDecoder/View/IMainView.cs

```
using MorseDecoder.Model;

namespace MorseDecoder.View;

/// <summary>Интерфейс главного представления, с которым работает
контроллер.</summary>
public interface IMainView
{
    /// <summary>Пользователь нажал «Ввести данные».</summary>
    event EventHandler? EnterDataRequested;

    /// <summary>Показывает окно ввода, заполненное <paramref
name="initial"/>; null — ввод отменён.</summary>
    MorseInput? RequestInput(MorseInput initial);

    void ShowError(string message);
}
```

Файл MorseDecoder/View/MainForm.cs

```
using MorseDecoder.Model;

namespace MorseDecoder.View;

/// <summary>
/// Главное окно (представление). Наблюдает за активной моделью: получив
уведомление
/// <see cref="ModelChanged"/>, само читает из модели данные и перерисовывает
метки.
/// </summary>
public sealed class MainForm : Form, IMainView, IMorseModelObserver
{
    private const string NoDataText = "— данные ещё не вводились —";

    private readonly MorseModel _model;

    private readonly TextBox _signalsBox;
    private readonly Label _alphabetLabel;
    private readonly TextBox _resultBox;
    private readonly Label _statsLabel;

    public event EventHandler? EnterDataRequested;

    public MainForm(MorseModel model)
    {
        _model = model;

        Text = "Декодер азбуки Морзе";
        Font = new Font("Segoe UI", 10F);
        AutoScaleMode = AutoScaleMode.Font;
        StartPosition = FormStartPosition.CenterScreen;
        ClientSize = new Size(620, 470);
        MinimumSize = new Size(480, 420);

        var layout = new TableLayoutPanel
        {
            Dock = DockStyle.Fill,
            Padding = new Padding(14),
            ColumnCount = 1,
            RowCount = 8
```

```
        };
        layout.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 100F));
        layout.RowStyles.Add(new RowStyle(SizeType.AutoSize));       //
заголовок
        layout.RowStyles.Add(new RowStyle(SizeType.AutoSize));       //
«Введённые сигналы»
        layout.RowStyles.Add(new RowStyle(SizeType.Percent, 40F));   //
сигналы
        layout.RowStyles.Add(new RowStyle(SizeType.AutoSize));       //
алфавит
        layout.RowStyles.Add(new RowStyle(SizeType.AutoSize));       //
«Результат»
        layout.RowStyles.Add(new RowStyle(SizeType.Percent, 60F));   //
результат
        layout.RowStyles.Add(new RowStyle(SizeType.AutoSize));       //
статистика
        layout.RowStyles.Add(new RowStyle(SizeType.AutoSize));       //
кнопка

        var titleLabel = new Label
        {
            Text = "Декодирование текста из азбуки Морзе",
            Font = new Font("Segoe UI", 14F, FontStyle.Bold),
            AutoSize = true,
            Margin = new Padding(3, 0, 3, 10)
        };

        _signalsBox = CreateReadOnlyBox(new Font("Consolas", 12F));
        _alphabetLabel = new Label { AutoSize = true, Margin = new Padding(3,
6, 3, 10) };
        _resultBox = CreateReadOnlyBox(new Font("Segoe UI", 14F,
FontStyle.Bold));
        _statsLabel = new Label { AutoSize = true, Margin = new Padding(3, 6,
3, 6) };

        var enterButton = new Button
        {
            Text = "Ввести данные",
            AutoSize = true,
            Padding = new Padding(10, 4, 10, 4),
            Anchor = AnchorStyles.Right
        };
        enterButton.Click += (_, _) => EnterDataRequested?.Invoke(this,
EventArgs.Empty);

        layout.Controls.Add(titleLabel, 0, 0);
        layout.Controls.Add(CreateCaption("Введённые сигналы:"), 0, 1);
        layout.Controls.Add(_signalsBox, 0, 2);
        layout.Controls.Add(_alphabetLabel, 0, 3);
        layout.Controls.Add(CreateCaption("Расшифрованный текст:"), 0, 4);
        layout.Controls.Add(_resultBox, 0, 5);
        layout.Controls.Add(_statsLabel, 0, 6);
        layout.Controls.Add(enterButton, 0, 7);
        Controls.Add(layout);

        AcceptButton = enterButton;

        _model.Attach(this);
        FormClosed += (_, _) => _model.Detach(this);
        ModelChanged();
    }
```

```
    public void ModelChanged()
    {
        if (!_model.HasResult)
        {
            _signalsBox.Text = NoDataText;
            _alphabetLabel.Text = "Алфавит: —";
            _resultBox.Text = NoDataText;
            _statsLabel.Text = "Нажмите «Ввести данные», чтобы ввести
сигналы.";
            return;
        }

        _signalsBox.Text = _model.Signals.Replace("\r\n", "\n").Replace("\n",
Environment.NewLine);
        _alphabetLabel.Text = $"Алфавит: {_model.Alphabet.ToDisplayName()}";
        _resultBox.Text = _model.DecodedText;
        _statsLabel.Text = $"Знаков: {_model.SymbolCount}    Слов:
{_model.WordCount}";
    }

    public MorseInput? RequestInput(MorseInput initial)
    {
        using var dialog = new InputForm(initial);
        return dialog.ShowDialog(this) == DialogResult.OK ? dialog.Result :
null;
    }

    public void ShowError(string message) =>
        MessageBox.Show(this, message, "Ошибка ввода", MessageBoxButtons.OK,
MessageBoxIcon.Error);

    private static Label CreateCaption(string text) => new()
    {
        Text = text,
        AutoSize = true,
        Font = new Font("Segoe UI", 10F, FontStyle.Bold),
        Margin = new Padding(3, 3, 3, 3)
    };

    private static TextBox CreateReadOnlyBox(Font font) => new()
    {
        Multiline = true,
        ReadOnly = true,
        WordWrap = true,
        ScrollBars = ScrollBars.Vertical,
        BackColor = SystemColors.Window,
        Font = font,
        Dock = DockStyle.Fill,
        TabStop = false
    };
}
```

Файл MorseDecoder/View/InputForm.cs

```
using MorseDecoder.Model;

namespace MorseDecoder.View;

/// <summary>Дополнительное окно ввода сигналов азбуки Морзе.</summary>
public sealed class InputForm : Form
```

```
{
    private const string RussianSample = ".--. .-. .. .-- . - / -- .. .-.";
// ПРИВЕТ МИР
    private const string LatinSample = ".... . .-.. .-.. --- / .-- --- .-. .-
.. -..";  // HELLO WORLD

    private readonly TextBox _signalsBox;
    private readonly RadioButton _russianRadio;
    private readonly RadioButton _latinRadio;

    public InputForm(MorseInput initial)
    {
        Text = "Ввод данных";
        Font = new Font("Segoe UI", 10F);
        AutoScaleMode = AutoScaleMode.Font;
        FormBorderStyle = FormBorderStyle.FixedDialog;
        StartPosition = FormStartPosition.CenterParent;
        MaximizeBox = false;
        MinimizeBox = false;
        ShowInTaskbar = false;
        ClientSize = new Size(560, 330);

        var layout = new TableLayoutPanel
        {
            Dock = DockStyle.Fill,
            Padding = new Padding(14),
            ColumnCount = 1,
            RowCount = 4
        };
        layout.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 100F));
        layout.RowStyles.Add(new RowStyle(SizeType.AutoSize));
        layout.RowStyles.Add(new RowStyle(SizeType.Percent, 100F));
        layout.RowStyles.Add(new RowStyle(SizeType.AutoSize));
        layout.RowStyles.Add(new RowStyle(SizeType.AutoSize));

        var hintLabel = new Label
        {
            Text = "Введите сигналы: точка «.», тире «-». Знаки разделяйте
пробелом, " +
                   "слова — косой чертой «/» или двумя пробелами.",
            AutoSize = true,
            MaximumSize = new Size(520, 0),
            Margin = new Padding(3, 0, 3, 8)
        };

        _signalsBox = new TextBox
        {
            Multiline = true,
            AcceptsReturn = false,
            ScrollBars = ScrollBars.Vertical,
            Font = new Font("Consolas", 12F),
            Dock = DockStyle.Fill,
            Text = initial.Signals
        };

        var alphabetGroup = new GroupBox
        {
            Text = "Алфавит",
            AutoSize = true,
            Dock = DockStyle.Fill,
            Padding = new Padding(8, 4, 8, 4)
        };
```

```
        var radioPanel = new FlowLayoutPanel { Dock = DockStyle.Fill,
AutoSize = true, WrapContents = false };
        _russianRadio = new RadioButton { Text = "Русский (кириллица)",
AutoSize = true, Margin = new Padding(3, 3, 24, 3) };
        _latinRadio = new RadioButton { Text = "Латинский", AutoSize = true
};
        radioPanel.Controls.AddRange(new Control[] { _russianRadio,
_latinRadio });
        alphabetGroup.Controls.Add(radioPanel);

        if (initial.Alphabet == MorseAlphabet.Latin)
            _latinRadio.Checked = true;
        else
            _russianRadio.Checked = true;

        var sampleButton = new Button { Text = "Пример", AutoSize = true,
Padding = new Padding(6, 2, 6, 2) };
        sampleButton.Click += (_, _) => _signalsBox.Text =
_latinRadio.Checked ? LatinSample : RussianSample;

        var okButton = new Button { Text = "Декодировать", AutoSize = true,
Padding = new Padding(6, 2, 6, 2), DialogResult = DialogResult.OK };
        var cancelButton = new Button { Text = "Отмена", AutoSize = true,
Padding = new Padding(6, 2, 6, 2), DialogResult = DialogResult.Cancel };

        var buttonsPanel = new FlowLayoutPanel
        {
            Dock = DockStyle.Fill,
            AutoSize = true,
            FlowDirection = FlowDirection.RightToLeft,
            Margin = new Padding(0, 8, 0, 0)
        };
        buttonsPanel.Controls.AddRange(new Control[] { cancelButton,
okButton, sampleButton });

        layout.Controls.Add(hintLabel, 0, 0);
        layout.Controls.Add(_signalsBox, 0, 1);
        layout.Controls.Add(alphabetGroup, 0, 2);
        layout.Controls.Add(buttonsPanel, 0, 3);
        Controls.Add(layout);

        AcceptButton = okButton;
        CancelButton = cancelButton;

        Shown += (_, _) =>
        {
            _signalsBox.Focus();
            _signalsBox.SelectAll();
        };
    }

    public MorseInput Result =>
        new(_signalsBox.Text, _latinRadio.Checked ? MorseAlphabet.Latin :
MorseAlphabet.Russian);
}
```

Файл MorseDecoder/Controller/MorseController.cs

```
using MorseDecoder.Model;
using MorseDecoder.View;
```

```
namespace MorseDecoder.Controller;

/// <summary>
/// Контроллер: принимает действия пользователя от представления и вызывает
методы модели.
/// Представление контроллер не обновляет — это делает активная модель через
уведомления.
/// </summary>
public sealed class MorseController
{
    private readonly MorseModel _model;
    private readonly IMainView _view;

    public MorseController(MorseModel model, IMainView view)
    {
        _model = model;
        _view = view;
        _view.EnterDataRequested += OnEnterDataRequested;
    }

    private void OnEnterDataRequested(object? sender, EventArgs e)
    {
        // В окне ввода восстанавливаются последние введённые данные.
        MorseInput input = _model.LastInput;

        while (true)
        {
            MorseInput? entered = _view.RequestInput(input);
            if (entered is null)
                return;

            try
            {
                _model.Decode(entered);
                return;
            }
            catch (MorseDecodeException ex)
            {
                _view.ShowError(ex.Message);
                // Повторно открываем окно с ошибочным вводом, чтобы его
можно было исправить.
                input = entered;
            }
        }
    }
}
```

ПРИЛОЖЕНИЕ Б
(справочное)
Текстовые модели UML и SysML v2

Файл docs/class-diagram.puml

```
@startuml MorseDecoder_MVC
title Декодер азбуки Морзе — MVC с активной моделью (диаграмма классов)
skinparam classAttributeIconSize 0
skinparam packageStyle rectangle

package Model {
  enum MorseAlphabet {
    Russian
    Latin
  }

  class MorseInput <<record>> {
    +Signals : string
    +Alphabet : MorseAlphabet
  }

  class DecodeResult <<record>> {
    +Text : string
    +SymbolCount : int
    +WordCount : int
  }

  interface IMorseModelObserver {
    +ModelChanged() : void
  }

  class MorseModel {
    -_observers : List<IMorseModelObserver>
    +Signals : string
    +Alphabet : MorseAlphabet
    +DecodedText : string
    +SymbolCount : int
    +WordCount : int
    +HasResult : bool
    +LastInput : MorseInput
    +Attach(o : IMorseModelObserver) : void
    +Detach(o : IMorseModelObserver) : void
    +Decode(input : MorseInput) : void
    -NotifyObservers() : void
  }

  class MorseTranslator <<static>> {
    +Decode(signals : string, alphabet : MorseAlphabet) : DecodeResult
    -Normalize(signals : string) : string
  }

  class MorseTable <<static>> {
    +TryGetSymbol(code : string, alphabet : MorseAlphabet, out symbol : char)
: bool
  }

  class MorseDecodeException
}
```

```
package View {
  interface IMainView {
    +EnterDataRequested : EventHandler
    +RequestInput(initial : MorseInput) : MorseInput
    +ShowError(message : string) : void
  }

  class MainForm {
    +ModelChanged() : void
    +RequestInput(initial : MorseInput) : MorseInput
    +ShowError(message : string) : void
  }

  class InputForm {
    +Result : MorseInput
  }
}

package Controller {
  class MorseController {
    -_model : MorseModel
    -_view : IMainView
    -OnEnterDataRequested(sender, e) : void
  }
}

MainForm ..|> IMainView
MainForm ..|> IMorseModelObserver
MainForm --> MorseModel : читает состояние
MainForm ..> InputForm : открывает

MorseModel o--> "*" IMorseModelObserver : уведомляет
MorseModel ..> MorseTranslator
MorseTranslator ..> MorseTable
MorseTranslator ..> MorseDecodeException : бросает
MorseTranslator ..> DecodeResult

MorseController --> MorseModel : Decode()
MorseController --> IMainView : подписка на события
@enduml
```

Файл docs/sequence-diagram.puml

```
@startuml MorseDecoder_Sequence
title Ввод данных и обновление представления (активная модель)

actor Пользователь
participant MainForm
participant MorseController
participant InputForm
participant MorseModel
participant MorseTranslator

Пользователь -> MainForm : нажимает «Ввести данные»
MainForm -> MorseController : EnterDataRequested
MorseController -> MorseModel : LastInput
MorseModel --> MorseController : последние введённые данные
loop пока ввод некорректен
  MorseController -> MainForm : RequestInput(input)
```

```
  MainForm -> InputForm : ShowDialog()
  Пользователь -> InputForm : вводит сигналы, выбирает алфавит
  InputForm --> MainForm : MorseInput
  MainForm --> MorseController : MorseInput
  MorseController -> MorseModel : Decode(input)
  MorseModel -> MorseTranslator : Decode(signals, alphabet)
  alt данные корректны
    MorseTranslator --> MorseModel : DecodeResult
    MorseModel -> MorseModel : сохранить состояние
    MorseModel -> MainForm : ModelChanged()
    MainForm -> MorseModel : чтение Signals, DecodedText, ...
    MainForm -> MainForm : обновить метки
  else ошибка
    MorseTranslator --> MorseController : MorseDecodeException
    MorseController -> MainForm : ShowError(message)
  end
end
@enduml
```

Файл docs/MorseDecoder.sysml

```
package MorseDecoderSystem {
    doc /* Лабораторная работа №1, вариант 16: утилита декодирования текста
из азбуки Морзе. */

    private import ScalarValues::*;

    // ---------- Типы данных ----------

    enum def MorseAlphabet {
        enum Russian;
        enum Latin;
    }

    attribute def MorseInput {
        attribute signals : String;
        attribute alphabet : MorseAlphabet;
    }

    attribute def DecodeResult {
        attribute text : String;
        attribute symbolCount : Integer;
        attribute wordCount : Integer;
    }

    // ---------- Порты ----------

    port def ActionPort {
        doc /* Действие пользователя «Ввести данные» (View -> Controller). */
        out item enterDataRequested;
    }

    port def DecodePort {
        doc /* Передача введённых данных в модель (Controller -> Model). */
        out item input : MorseInput;
    }

    port def NotifyPort {
        doc /* Уведомление активной модели об изменении состояния (Model ->
View). */
```

```
        out item modelChanged;
    }

    // ---------- Структура ----------

    part def MorseModel {
        attribute lastInput : MorseInput;
        attribute result : DecodeResult;
        port decode : ~DecodePort;
        port notify : NotifyPort;
    }

    part def MainForm {
        port actions : ActionPort;
        port notify : ~NotifyPort;
    }

    part def InputForm;

    part def MorseController {
        port actions : ~ActionPort;
        port decode : DecodePort;
    }

    part def Application {
        part model : MorseModel;
        part mainView : MainForm;
        part inputView : InputForm;
        part controller : MorseController;

        connect mainView.actions to controller.actions;
        connect controller.decode to model.decode;
        connect model.notify to mainView.notify;
    }

    // ---------- Поведение ----------

    action def DecodeMorse {
        in input : MorseInput;
        out result : DecodeResult;

        action normalize;
        action splitWords;
        action lookupCodes;
        action notifyObservers;

        first normalize then splitWords;
        first splitWords then lookupCodes;
        first lookupCodes then notifyObservers;
    }

    // ---------- Требования варианта ----------

    requirement def DecodeRequirement {
        doc /* Программа преобразует введённые сигналы азбуки Морзе в текст.
*/
    }

    requirement def SeparateInputWindowRequirement {
        doc /* Данные вводятся в дополнительном окне, открываемом кнопкой
«Ввести данные». */
    }
```

```
    requirement def RestoreLastInputRequirement {
        doc /* Последние введённые данные запоминаются и восстанавливаются
при повторном вводе. */
    }

    requirement def ShowResultRequirement {
        doc /* Результат декодирования отображается на главной форме. */
    }

    requirement def InvalidInputRequirement {
        doc /* При вводе некорректных данных выводится сообщение об ошибке.
*/
    }

    requirement def ActiveModelRequirement {
        doc /* Приложение построено по схеме MVC с активной моделью. */
    }

    requirement decode : DecodeRequirement;
    requirement separateInputWindow : SeparateInputWindowRequirement;
    requirement restoreLastInput : RestoreLastInputRequirement;
    requirement showResult : ShowResultRequirement;
    requirement invalidInput : InvalidInputRequirement;
    requirement activeModel : ActiveModelRequirement;

    part variant16 : Application;

    satisfy decode by variant16;
    satisfy separateInputWindow by variant16;
    satisfy restoreLastInput by variant16;
    satisfy showResult by variant16;
    satisfy invalidInput by variant16;
    satisfy activeModel by variant16;
}
```